

Python Programming

Module 1: Python Introduction

1. What is Python?

- **Python** is a high-level, general-purpose programming language.
- It is **easy to learn, read, and write**.
- Python supports **object-oriented, procedural, and functional programming**.
- Python uses **indentation** to define blocks of code.
- Python files generally use the .py extension.

Example:

```
print("Hello, World!")
```

2. History of Python

- Python was created by **Guido van Rossum**.
 - Development started in the **late 1980s**.
 - Python was first released in **1991**.
 - Python 2.0 was released in **2000**.
 - Python 3.0 was released in **2008**.
 - Python 3 is the modern and recommended version.
-

3. Features of Python

- **Easy to learn** – Simple and readable syntax.
 - **High-level** – No need to manage low-level memory directly.
 - **Interpreted** – Python programs are executed by a Python implementation.
 - **Dynamically typed** – Variable types are determined at runtime.
 - **Object-oriented** – Supports classes and objects.
 - **Portable** – Programs can run on different operating systems.
 - **Open source** – Free to use and modify.
 - **Large standard library** – Provides many built-in modules.
 - **Extensible** – Can work with languages such as C/C++.
 - **Large community** – Many libraries, frameworks, and learning resources are available.
-

4. Advantages of Python

- Simple and readable syntax.

- Easy for beginners.
- Faster development.
- Large number of libraries and frameworks.
- Cross-platform.
- Supports multiple programming paradigms.
- Useful for automation, web development, AI, ML, and data science.
- Large developer community.

Disadvantages of Python

- Generally slower than compiled languages such as C/C++.
 - Higher memory consumption in many applications.
 - Not usually preferred for mobile application development.
 - Dynamic typing can cause some errors to appear only at runtime.
 - Not ideal for some low-level/system programming tasks.
-

5. Applications of Python

Python is used in:

- **Web Development** – Django, Flask, FastAPI.
 - **Data Science** – NumPy, Pandas.
 - **Machine Learning** – Scikit-learn.
 - **Artificial Intelligence** – PyTorch, TensorFlow.
 - **Automation/Scripting** – Automating repetitive tasks.
 - **Data Visualization** – Matplotlib, Seaborn.
 - **Web Scraping** – BeautifulSoup, Scrapy.
 - **Desktop Applications** – Tkinter, PyQt.
 - **Testing** – pytest, unittest.
 - **Cybersecurity and Networking.**
 - **Scientific Computing.**
-

6. Python vs C/C++/Java

Feature	Python	C	C++	Java
Level	High-level	Low-level	Middle/High-level	High-level
Syntax	Simple	Complex	Complex	Moderate
Typing	Dynamic	Static	Static	Static
Execution	Python implementation	Compiled	Compiled	JVM

Feature	Python	C	C++	Java
Speed	Generally slower	Fast	Very fast	Fast
Memory management	Automatic	Manual	Manual/Automatic	Garbage Collector
Beginner friendly	Very high	Moderate	Moderate	Moderate
Main uses	AI, ML, Web, Data	System programming	Systems, Games	Enterprise, Android

Simple idea: Python focuses more on **simplicity and productivity**, while C/C++ generally provide more **control and performance**.

7. Python Versions

Python has two major generations:

Python 2

- Older generation.
- Officially discontinued in **2020**.
- Should not be used for new projects.

Python 3

- Modern Python generation.
- Actively developed.
- Recommended for new projects.

Important: Python 2 and Python 3 have some syntax and library differences.

8. Python Implementations

A **Python implementation** is a program that executes Python code.

8.1 CPython

- The **standard and most widely used Python implementation**.
- Written mainly in **C**.
- Usually what people mean when they install Python from python.org.

8.2 PyPy

- Alternative Python implementation.
- Uses **JIT (Just-In-Time) compilation**.
- Can be faster for some long-running Python programs.

8.3 Jython

- Python implementation designed to run on the **Java Virtual Machine (JVM)**.
- Allows Python code to interact with Java libraries.

8.4 IronPython

- Python implementation for the **.NET ecosystem**.
- Allows Python programs to work with .NET libraries.

9. Python Interpreter

- A **Python interpreter** is the software that executes Python programs.
- It reads Python code and performs the required operations.
- It also reports errors when the program contains invalid code.

Example:

```
print("Hello")
```

The Python implementation executes this statement and displays:

Hello

10. Python Execution Process

The basic execution flow is:

```
Python Source Code
    ↓
  Bytecode
    ↓
Python Virtual Machine (PVM)
    ↓
    Output
```

Step 1: Source Code

- Programmer writes Python code in a .py file.

Step 2: Bytecode

- Python implementation converts Python source into **bytecode**.
- Bytecode is an intermediate representation.

Step 3: Python Virtual Machine

- The **PVM** executes the bytecode.

Step 4: Output

- The program produces the required output.

Note: The exact internal behavior can vary between Python implementations, but this model is useful for understanding standard CPython execution.

11. Source Code → Bytecode → Python Virtual Machine

Source Code

Human-readable Python program.

```
x = 10  
print(x)
```

Bytecode

- Intermediate instructions generated from Python code.
- In CPython, bytecode may be stored in .pyc files.

Python Virtual Machine (PVM)

- Executes the Python bytecode.
- It is part of the Python runtime.

Flow:

```
.py file  
↓  
Source Code  
↓  
Bytecode  
↓  
PVM
```

↓
Output

12. Python Installation

Basic steps:

1. Download Python from the official Python website.
2. Install Python.
3. Make sure Python is added to the system PATH on Windows if desired.
4. Verify installation.

Check Python version:

```
python --version
```

or:

```
python3 --version
```

Check pip:

```
pip --version
```

pip is Python's package installer.

13. Python IDEs and Editors

VS Code

- Lightweight source-code editor.
- Supports Python through extensions.
- Good for general Python development.

PyCharm

- Full-featured Python IDE.
- Provides debugging, project management, code completion, and other development tools.

Jupyter Notebook

- Interactive environment.
- Code is written and executed in **cells**.

- Popular for **data science, ML, visualization, and experimentation.**
-

14. Running Python Programs

There are several ways to run Python.

Using Python file

Create:

hello.py

Write:

```
print("Hello Python")
```

Run from terminal:

```
python hello.py
```

Using IDE

- Open the .py file.
- Click **Run**.
- View the output in the terminal/output window.

Using Jupyter Notebook

- Write code inside a cell.
 - Execute the cell.
 - Output appears below the cell.
-

15. Interactive Mode vs Script Mode

Interactive Mode

- Python commands are executed **one at a time**.
- Useful for learning, testing, and quick experiments.
- Python displays the >>> prompt.

Example:

```
>>> 10 + 20
30
>>> print("Hello")
Hello
```

Script Mode

- Python code is saved in a .py file.
- The complete file can be executed.
- Best for developing and maintaining programs.

Example:

hello.py

```
name = "Prathamesh"
print("Hello", name)
```

Run:

```
python hello.py
```

Module 2: Python Basics

2.1 Python Syntax

1. Python Syntax

- **Syntax** means the rules for writing Python code.
- Python syntax is simple and readable.
- Python is **case-sensitive**.

```
name = "Prathamesh"
print(name)
```

2. Indentation

- **Indentation** means spaces at the beginning of a line.
- Python uses indentation to define **blocks of code**.
- Usually, **4 spaces** are used.

```
if age >= 18:
    print("Adult")
```

- Incorrect indentation can cause an IndentationError.

3. Comments

- Comments are notes written in code for understanding.
- Python ignores comments during execution.

Single-line comment:

```
# This is a comment  
print("Hello")
```

Multi-line documentation/comment style:

```
"""  
This is a  
multi-line text.  
"""
```

4. Statements

- A **statement** is an instruction that Python executes.

```
x = 10  
print(x)
```

Here:

- `x = 10` → assignment statement
 - `print(x)` → function call statement
-

5. Expressions

- An **expression** is a combination of values, variables, operators, and function calls that produces a value.

10 + 20

Result:

30

Example:

```
x = 10  
y = x + 5
```

$x + 5$ is an expression.

6. Keywords

- **Keywords** are reserved words with special meaning in Python.
- They cannot normally be used as variable or function names.

Examples:

```
if  
else  
for  
while  
def  
class  
return  
import  
True  
False  
None
```

You can view Python keywords using:

```
import keyword  
  
print(keyword.kwlist)
```

7. Identifiers

- An **identifier** is the name given to a variable, function, class, module, etc.

```
student_name = "Prathamesh"
```

Here, `student_name` is an identifier.

Rules:

- Can contain letters, digits, and `_`.
- Cannot start with a digit.
- Cannot be a keyword.

- Python identifiers are case-sensitive.

Valid:

```
name
student_name
age1
_total
```

Invalid:

```
1name
student-name
class
```

8. Naming Conventions

Naming conventions make code easier to read.

Element	Convention	Example
Variable	snake_case	student_name
Function	snake_case	calculate_total()
Class	PascalCase	StudentDetails
Constant	UPPER_CASE	MAX_SIZE
Private/internal name	_name	_salary

2.2 Variables

1. Variables

- A **variable** is a name that refers to a value/object.
- Variables are used to store and work with data.

```
name = "Prathamesh"
age = 22
```

2. Variable Declaration

- Python does **not require a separate variable declaration**.
- A variable is created when a value is assigned to it.

```
x = 10  
name = "Python"
```

3. Dynamic Typing

- Python is **dynamically typed**.
- You don't need to specify the variable's data type.
- The type is determined at runtime.
- A variable can later refer to an object of another type.

```
x = 10  
x = "Python"
```

Initially x refers to an integer, and later it refers to a string.

4. Multiple Assignment

- Multiple variables can be assigned values in one statement.

```
a, b, c = 10, 20, 30
```

Now:

```
a = 10  
b = 20  
c = 30
```

5. Multiple Variables with Same Value

- Multiple variables can also receive the same value.

```
a = b = c = 100
```

All three variables refer to 100.

6. Variable Swapping

Python allows swapping variables without a temporary variable.

```
a = 10
```

```
b = 20
```

```
a, b = b, a
```

Result:

```
a = 20
```

```
b = 10
```

7. Constants

- A **constant** is a value that should not be changed during the program.
- Python does not enforce constants directly.
- By convention, constants are written in **uppercase**.

```
PI = 3.14159
```

```
MAX_USERS = 100
```

Python allows these values to be reassigned, so the uppercase name is only a convention.

8. id()

- `id()` returns the identity of an object.
- The returned integer is unique for that object during its lifetime.

```
x = 10
```

```
print(id(x))
```

Purpose: Mainly useful for understanding object identity.

9. type()

- `type()` returns the type/class of an object.

```
x = 10
```

```
print(type(x))
```

Output:

```
<class 'int'>
```

Example:

```
name = "Python"  
print(type(name))
```

Output:

```
<class 'str'>
```

2.3 Input and Output

1. print()

- print() is used to display output on the screen.

```
print("Hello Python")
```

Multiple values:

```
name = "Prathamesh"  
age = 22
```

```
print(name, age)
```

Output:

```
Prathamesh 22
```

2. input()

- input() is used to receive input from the user.
- It **always returns a string**.

```
name = input("Enter your name: ")
```

```
print(name)
```

For numeric input, conversion is required:

```
age = int(input("Enter your age: "))
```

3. Formatting Output

Output formatting means presenting data in a readable way.

Using f-strings:

```
name = "Prathamesh"  
age = 22  
  
print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Prathamesh and I am 22 years old.

Using format():

```
print("My name is {} and I am {} years old.".format(name, age))
```

4. sep

- sep specifies the separator between multiple values in print().
- Default separator is a space.

```
print("Python", "Java", "C++")
```

Output:

Python Java C++

Using sep:

```
print("Python", "Java", "C++", sep=" | ")
```

Output:

Python | Java | C++

5. end

- end specifies what should be printed at the end of print().
- Default value is a newline (\n).

```
print("Hello", end=" ")  
print("Python")
```

Output:

Hello Python

Another example:

```
print("A", end="-")  
print("B")
```

Output:

A-B

6. Escape Characters

- Escape characters are special character sequences beginning with \.
- They are used to represent special formatting characters.

Escape	Meaning
\n	New line
\t	Tab
\\	Backslash
\'	Single quote
\"	Double quote

Module 3: Python Data Types

Data types define what kind of value an object represents and what operations can be performed on it.

3.1 Basic Data Types

1. int

- Used to represent **whole numbers**.
- Can be positive, negative, or zero.

```
age = 22  
marks = -10  
print(type(age))  
# <class 'int'>
```


2. float

- Used for **decimal/floating-point numbers**.

```
price = 99.50
temperature = -2.5
print(type(price))
# <class 'float'>
```

3. complex

- Used to represent **complex numbers**.
- Written using j for the imaginary part.

```
z = 3 + 4j
```

- 3 → real part
- 4 → imaginary part

```
print(z.real)
print(z.imag)
```

4. bool

- Represents **True or False** values.
- Mainly used in conditions and logical operations.

```
is_student = True
is_logged_in = False
print(type(is_student))
# <class 'bool'>
```

5. str

- Used to represent **text/string data**.
- Strings can be written using single, double, or triple quotes.

```
name = "Prathamesh"
language = 'Python'
```

Example:

```
message = "Hello Python"  
print(message)
```

6. NoneType

- None represents the **absence of a value** or "no value".
- Its type is NoneType.

```
result = None
```

```
print(type(result))  
# <class 'NoneType'>
```

Commonly used when a value is not available yet.

3.2 Collection Data Types

1. List

- A **list** stores multiple values.
- Ordered.
- Mutable.
- Allows duplicate values.

```
numbers = [10, 20, 30, 20]
```

Example:

```
numbers.append(40)  
print(numbers)
```

2. Tuple

- A **tuple** stores multiple values.
- Ordered.
- Immutable.
- Allows duplicate values.

```
numbers = (10, 20, 30, 20)
```

Once created, its elements cannot normally be changed.

3. Set

- A **set** stores unique values.
- Mutable.
- Does not support duplicate elements.
- Set iteration order should not be relied upon.

```
numbers = {10, 20, 30, 20}
```

```
print(numbers)
```

Duplicate 20 is removed.

4. Dictionary

- Stores data in **key-value pairs**.
- Mutable.
- Keys must be hashable.
- In modern Python, dictionaries preserve **insertion order**.

```
student = {  
    "name": "Prathamesh",  
    "age": 22,  
    "course": "AI & DS"  
}
```

Access a value:

```
print(student["name"])
```

Output:

Prathamesh

3.3 Mutable vs Immutable

Mutable

- Object can be **changed after creation**.

Examples:

- list
- set
- dict

```
numbers = [10, 20]  
numbers.append(30)
```

The same list object can be modified.

Immutable

- Object **cannot be changed after creation.**

Examples:

- int
- float
- complex
- bool
- str
- tuple
- None

```
name = "Python"
```

You cannot modify an individual character of the string.

Quick Table

Type	Mutable?
int	No
float	No
complex	No
bool	No
str	No
tuple	No
list	Yes
set	Yes
dict	Yes

3.4 Ordered vs Unordered

Ordered

- Elements have a defined sequence that can be accessed/iterated consistently.

Examples:

- str
- list
- tuple
- dict → preserves insertion order

```
numbers = [10, 20, 30]
```

Set

- A set is **not sequence-ordered**.
- Do not depend on the order in which set elements are displayed or iterated.

```
numbers = {30, 10, 20}
```

3.5 Hashable vs Unhashable

Hashable

An object is **hashable** if it has a hash value that remains stable during its lifetime and can be used as a dictionary key or set element.

Examples:

- int
- float
- str
- bool
- tuple containing only hashable elements

```
student = {  
    "name": "Prathamesh"  
}
```

"name" is hashable, so it can be a dictionary key.

Unhashable

Objects that cannot be used as dictionary keys or set elements.

Examples:

- list
- set
- dict

```
# Invalid  
data = {[1, 2]: "value"}
```

3.6 Type Conversion

Type conversion means changing a value from one data type to another.

Common functions:

```
int()  
float()  
str()  
bool()  
list()  
tuple()  
set()  
dict()
```

Example:

```
x = "100"  
  
y = int(x)  
  
print(y)  
print(type(y))
```

Output:

```
100  
<class 'int'>
```

3.7 Type Casting

In beginner-level Python discussions, **type casting** and **type conversion** are often used interchangeably.

Example:

```
x = 10
y = float(x)
```

```
print(y)
# 10.0
```

Common conversions

```
int("10")    # 10
float("10.5") # 10.5
str(100)     # "100"
bool(1)      # True
```

Conversion can fail if the value is not suitable, for example `int("hello")` raises `ValueError`.

3.8 type()

- `type()` returns the type/class of an object.

```
x = 10
```

```
print(type(x))
```

Output:

```
<class 'int'>
```

More examples:

```
print(type("Python")) # str
print(type(10.5))     # float
print(type([1, 2, 3])) # list
```

3.9 isinstance()

- `isinstance()` checks whether an object belongs to a specified type or class.
- Returns `True` or `False`.

```
x = 10
```

```
print(isinstance(x, int))
```

Output:

True

Example:

```
name = "Python"
```

```
print(isinstance(name, str))
```

```
# True
```

type() vs isinstance()

```
type(x) == int
```

checks the exact type.

```
isinstance(x, int)
```

checks whether the object is an instance of that type or a compatible subclass.

Module 4: Python Operators

Operators are symbols or keywords used to perform operations on values and variables.

4.1 Arithmetic Operators

Used for mathematical operations.

Operator	Meaning	Example
+	Addition	$10 + 3 \rightarrow 13$
-	Subtraction	$10 - 3 \rightarrow 7$
*	Multiplication	$10 * 3 \rightarrow 30$
/	Division	$10 / 3 \rightarrow 3.333...$
//	Floor division	$10 // 3 \rightarrow 3$
%	Modulus/remainder	$10 \% 3 \rightarrow 1$
**	Exponentiation	$2 ** 3 \rightarrow 8$

Example:

```
a = 10
```

```
b = 3
```

```
print(a + b)
```



```
print(a % b)
print(a ** b)
```

4.2 Comparison Operators

Used to compare values.

The result is True or False.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
a = 10
b = 20
```

```
print(a < b)
```

Output:

True

4.3 Assignment Operators

Used to assign or update values.

Operator	Example	Equivalent
=	x = 10	Assign
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
//=	x //= 5	x = x // 5
%=	x %= 5	x = x % 5

Operator	Example	Equivalent
**=	x **= 5	x = x ** 5

Example:

```
x = 10  
x += 5
```

```
print(x)  
# 15
```

4.4 Logical Operators

Used to combine or modify conditions.

and

Returns True when both conditions are true.

```
age = 22
```

```
print(age > 18 and age < 30)  
# True
```

or

Returns True when at least one condition is true.

```
print(age < 18 or age > 20)  
# True
```

not

Reverses the truth value.

```
print(not True)  
# False
```

4.5 Bitwise Operators

Used to perform operations on the **bits** of integer values.

Operator	Meaning
----------	---------

Operator	Meaning
&	AND
	OR
^	XOR
~	NOT
<<	Left shift
>>	Right shift

Example:

```
a = 5
b = 3
```

```
print(a & b)
```

Binary:

```
5 = 101
3 = 011
---
& 001 = 1
```

Output:

```
1
```

4.6 Membership Operators

Used to check whether a value exists inside a collection or sequence.

in

Returns True if the value exists.

```
numbers = [10, 20, 30]
```

```
print(20 in numbers)
# True
```

not in

Returns True if the value does not exist.

```
print(50 not in numbers)
# True
```

Can also be used with strings:

```
print("Py" in "Python")
# True
```

4.7 Identity Operators

Identity operators check whether two references refer to the **same object**.

is

```
a = [1, 2]
b = a
```

```
print(a is b)
# True
```

Both variables refer to the same list object.

is not

```
a = [1, 2]
b = [1, 2]
```

```
print(a is not b)
# True
```

The two lists have equal contents but are different objects.

4.8 == vs is

This is **very important**.

==

Checks **value equality**.

```
a = [1, 2]
b = [1, 2]
```

Prathamesh Arvind Jadhav

```
print(a == b)
# True
```

Both lists contain the same values.

is

Checks **object identity**.

```
print(a is b)
# False
```

They are two different list objects.

Remember

`==` → Are the values equal?

`is` → Are they the same object?

Use `==` when comparing values.

Use `is` mainly for identity checks, especially:

if value is `None`:

...

4.9 Operator Precedence

Operator precedence determines which operator is evaluated first.

Example:

```
result = 10 + 2 * 3
```

Multiplication is performed first:

```
10 + (2 * 3)
```

```
10 + 6
```

```
16
```

Simplified precedence order

()

**

+x, -x, ~x

*, /, //, %
+, -
<, <=, >, >=, ==, !=
not
and
or

When in doubt, use parentheses:

result = (10 + 2) * 3

4.10 Associativity

Associativity determines the evaluation direction when operators have the same precedence.

Most operators are evaluated **left to right**.

Example:

10 - 5 - 2

Evaluated as:

(10 - 5) - 2
= 3

Important exception

Exponentiation ** is evaluated **right to left**:

2 ** 3 ** 2

Means:

2 ** (3 ** 2)
= 2 ** 9
= 512

Module 5: Strings

A **string** is a sequence of characters used to store text. Strings are one of the most commonly used data types in Python.

5.1 Creating Strings

Strings can be created by assigning text to a variable.

```
name = "Prathamesh"  
language = 'Python'
```

A string can contain letters, numbers, spaces, and special characters.

```
message = "Python 3 is easy!"
```

5.2 Single, Double, and Triple Quotes

Single Quotes

```
name = 'Python'
```

Double Quotes

```
name = "Python"
```

Both create a string.

Triple Quotes

Used mainly for **multi-line strings** and documentation strings.

```
message = """This is  
a multi-line  
string."""
```

You can also use:

```
message = """This is  
a multi-line  
string."""
```

5.3 String Indexing

- Indexing is used to access individual characters.
- Python indexing starts from **0**.

```
text = "Python"
```

Character P y t h o n

Character P y t h o n

Index 0 1 2 3 4 5

Example:

```
print(text[0]) # P
print(text[2]) # t
print(text[5]) # n
```

5.4 Negative Indexing

- Negative indexing starts from the **end** of the string.
- -1 represents the last character.

```
text = "Python"
```

```
print(text[-1]) # n
print(text[-2]) # o
print(text[-6]) # P
P y t h o n
0 1 2 3 4 5
-6 -5 -4 -3 -2 -1
```

5.5 String Slicing

Slicing is used to extract a portion of a string.

Syntax

```
string[start:stop:step]
```

- start → starting index
- stop → ending index, **not included**
- step → number of positions to move

Example:

```
text = "Python"

print(text[0:3])
```

Output:

Pyt

More examples

```
print(text[:3]) # Pyt
print(text[2:]) # thon
print(text[::2]) # Pto
print(text[::-1]) # nohtyP
```

5.6 String Concatenation

- Concatenation means **joining strings together**.
- The + operator is used.

```
first = "Hello"
second = "Python"
```

```
result = first + " " + second
```

```
print(result)
```

Output:

Hello Python

5.7 String Repetition

- The * operator repeats a string.

```
text = "Python "
```

```
print(text * 3)
```

Output:

Python Python Python

5.8 String Immutability

- Strings are **immutable**.
- This means an existing string cannot be changed directly.

```
text = "Python"
```

```
# text[0] = "J" # TypeError
```

Prathamesh Arvind Jadhav

Instead, create a new string:

```
text = "J" + text[1:]
```

```
print(text)
```

Output:

Jython

Remember:

String → Immutable

List → Mutable

5.9 String Methods

upper()

Converts all characters to uppercase.

```
text = "python"
```

```
print(text.upper())
```

Output:

PYTHON

lower()

Converts all characters to lowercase.

```
text = "PYTHON"
```

```
print(text.lower())
```

Output:

python

capitalize()

Converts the first character to uppercase and the remaining characters to lowercase.

```
text = "python programming"
```

```
print(text.capitalize())
```

Output:

Python programming

title()

Converts the first character of each word to uppercase.

```
text = "python programming"
```

```
print(text.title())
```

Output:

Python Programming

strip()

Removes whitespace from the beginning and end.

```
text = " Python "
```

```
print(text.strip())
```

Output:

Python

Related methods:

```
text.lstrip() # Remove from left
```

```
text.rstrip() # Remove from right
```

replace()

Replaces one substring with another.

Prathamesh Arvind Jadhav

```
text = "I like Java"  
print(text.replace("Java", "Python"))
```

Output:

I like Python

split()

Splits a string into a **list**.

```
text = "Python is easy"
```

```
words = text.split()
```

```
print(words)
```

Output:

```
['Python', 'is', 'easy']
```

Using a separator:

```
data = "apple,banana,mango"
```

```
print(data.split(","))
```

join()

Joins elements of an iterable into a string.

```
words = ["Python", "is", "easy"]
```

```
result = " ".join(words)
```

```
print(result)
```

Output:

Python is easy

Remember:

`split() → String → List`
`join() → List/iterable → String`

find()

Returns the index of the first occurrence of a substring.

```
text = "Python programming"
print(text.find("program"))
```

Output:

7

If not found:

```
print(text.find("Java"))
```

Output:

-1

index()

Returns the index of the first occurrence.

```
text = "Python programming"
print(text.index("program"))
```

Output:

7

Difference:

`find() → returns -1 if not found`
`index() → raises ValueError if not found`

count()

Counts how many times a substring occurs.

Prathamesh Arvind Jadhav

```
text = "banana"  
print(text.count("a"))
```

Output:

3

startswith()

Checks whether a string starts with a specified value.

```
text = "Python Programming"  
print(text.startswith("Python"))
```

Output:

True

endswith()

Checks whether a string ends with a specified value.

```
text = "Python.py"  
print(text.endswith(".py"))
```

Output:

True

5.10 String Formatting

% Formatting

Older style of string formatting.

```
name = "Prathamesh"  
age = 22  
  
print("My name is %s and I am %d years old." % (name, age))
```

Common format specifiers:

%s → string
%d → integer
%f → float

str.format()

Uses {} placeholders.

```
name = "Prathamesh"  
age = 22
```

```
print("My name is {} and I am {} years old.".format(name, age))
```

Can also use named placeholders:

```
print("My name is {name} and I am {age}.".format(  
    name=name,  
    age=age  
))
```

f-Strings

- Modern and highly readable way to format strings.
- Available from Python 3.6+.

```
name = "Prathamesh"  
age = 22
```

```
print(f"My name is {name} and I am {age} years old.")
```

Expressions can also be used:

```
a = 10  
b = 20
```

```
print(f"Sum = {a + b}")
```

For modern Python code, f-strings are generally the preferred formatting style.

Module 6: Conditional Statements

Conditional statements allow a program to make decisions based on conditions.

6.1 if Statement

- Executes a block of code only when the condition is True.

Syntax

```
if condition:  
    statement
```

Example:

```
age = 20
```

```
if age >= 18:  
    print("Eligible to vote")
```

Important: The code inside if must be properly indented.

6.2 if-else

- if executes when the condition is true.
- else executes when the condition is false.

```
age = 16
```

```
if age >= 18:  
    print("Adult")  
else:  
    print("Minor")
```

Output:

```
Minor
```

6.3 if-elif-else

Used when there are **multiple conditions**.

```
marks = 75
```



```
if marks >= 90:
    print("A")
elif marks >= 75:
    print("B")
elif marks >= 60:
    print("C")
else:
    print("D")
```

Python checks conditions **from top to bottom**.

Once a condition is true, its block executes and the remaining elif/else blocks are skipped.

6.4 Nested if

- An if statement inside another if statement is called a **nested if**.

```
age = 22
has_id = True
```

```
if age >= 18:
    if has_id:
        print("Entry allowed")
```

Nested conditions are useful when one condition depends on another.

6.5 Conditional Expression / Ternary Operator

- A conditional expression is a short way to write a simple if-else.

Normal form

```
if age >= 18:
    result = "Adult"
else:
    result = "Minor"
```

Conditional expression

```
result = "Adult" if age >= 18 else "Minor"
```

Syntax:

value_if_true if condition else value_if_false

Use it for **simple conditions**; use normal if-else when the logic is complex.

Module 7: Loops

Loops are used to execute a block of code repeatedly. They are essential for processing collections, solving coding problems, and automating repetitive tasks.

7.1 for Loop

A for loop is generally used when you want to iterate over a sequence or iterable such as a list, string, tuple, or range.

Basic for Loop

```
for item in [10, 20, 30]:  
    print(item)
```

Output:

```
10  
20  
30
```

With a string:

```
for char in "Python":  
    print(char)
```

Each character is processed one by one.

7.2 range()

range() generates a sequence of numbers, commonly used with for.

range(stop)

```
for i in range(5):  
    print(i)
```

Output:

0
1
2
3
4

The stop value is **not included**.

range(start, stop)

```
for i in range(2, 6):  
    print(i)
```

Output:

2
3
4
5

range(start, stop, step)

```
for i in range(2, 10, 2):  
    print(i)
```

Output:

2
4
6
8

Reverse range

```
for i in range(5, 0, -1):  
    print(i)
```

Output:

5
4
3
2
1

7.3 Nested for Loop

A loop inside another loop is called a **nested loop**.

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```

Nested loops are commonly used for:

- Patterns
- Matrices
- 2D lists
- Comparing elements

Example:

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print("*", end=" ")  
    print()
```

Output:

```
* * *  
* * *  
* * *
```

7.4 while Loop

A while loop repeatedly executes code **while a condition is true**.

Basic Syntax

```
while condition:  
    statement
```

Example:

```
i = 1
```

```
while i <= 5:  
    print(i)
```

```
i += 1
```

Output:

```
1
2
3
4
5
```

Make sure the condition eventually becomes False; otherwise, the loop may continue indefinitely.

7.5 Infinite Loops

An **infinite loop** is a loop that never stops because its condition never becomes false.

Example:

```
while True:
    print("Hello")
```

This keeps running until the program is interrupted.

A common mistake:

```
i = 1

while i <= 5:
    print(i)
```

Here, *i* is never updated, so the condition remains true.

Correct:

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

7.6 Nested while Loop

A while loop can be placed inside another while loop.

```
i = 1

while i <= 3:
    j = 1

    while j <= 2:
        print(i, j)
        j += 1

    i += 1
```

Nested while loops are useful for problems involving multiple levels of repetition.

7.7 break

break immediately **terminates the nearest enclosing loop**.

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

Output:

```
1
2
3
4
```

7.8 continue

continue skips the **remaining code in the current iteration** and moves to the next iteration.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output:

```
1
```

2
4
5

7.9 pass

pass does nothing.

It is used as a **placeholder** when Python requires a statement but you do not want to execute anything yet.

```
for i in range(5):  
    pass
```

Example:

```
def my_function():  
    pass
```

Later, you can replace pass with actual code.

Remember

break → Stop the loop
continue → Skip current iteration
pass → Do nothing

7.10 for vs while

for Loop	while Loop
Usually used to iterate over an iterable	Usually used while a condition remains true
Common when the iteration is naturally sequence-based	Common when the stopping condition is dynamic
Often used with range()	Requires a condition
Less likely to accidentally become infinite	Must update state carefully to avoid infinite loops

Example:

```
for i in range(5):  
    print(i)
```

vs.

```
i = 0
```

```
while i < 5:  
    print(i)  
    i += 1
```

Both can produce the same result.

7.11 Loop else

Python allows an else block with for and while loops.

The else block executes when the loop **finishes normally**, meaning it was not terminated by break.

for with else

```
for i in range(5):  
    print(i)  
else:  
    print("Loop completed")
```

Output:

```
0  
1  
2  
3  
4  
Loop completed
```

break with else

```
for i in range(5):  
    if i == 3:  
        break  
else:  
    print("Loop completed")
```

Here, the else block does **not** execute because break terminated the loop.

Common use: Searching

```
numbers = [10, 20, 30, 40]
```



```
for num in numbers:
    if num == 30:
        print("Found")
        break
else:
    print("Not found")
```

Module 8: Lists

A **list** is an ordered, mutable collection that can store multiple values of different types. Lists are one of the most important Python data structures.

8.1 Creating Lists

Lists are created using square brackets [].

```
numbers = [10, 20, 30, 40]
```

A list can contain different data types:

```
data = [10, "Python", 3.14, True]
```

An empty list:

```
items = []
```

8.2 List Indexing

- List indexing starts from 0.
- Negative indexing starts from -1.

```
numbers = [10, 20, 30, 40]
Value:  10  20  30  40
Index:   0   1   2   3
Negative:-4 -3 -2 -1
```

Example:

```
print(numbers[0]) # 10
print(numbers[2]) # 30
print(numbers[-1]) # 40
```

8.3 List Slicing

Syntax:

```
list[start:stop:step]
```

Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

Other examples:

```
print(numbers[:3])    # [10, 20, 30]
print(numbers[2:])    # [30, 40, 50]
print(numbers[::2])   # [10, 30, 50]
print(numbers[::-1])  # [50, 40, 30, 20, 10]
```

8.4 Nested Lists

A list can contain other lists.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Access an element:

```
print(matrix[0][1])
```

Output:

```
2
```

Nested lists are commonly used for:

- Matrices
- Tables

- 2D data
 - Grid-based problems
-

8.5 List Mutability

Lists are **mutable**, meaning their contents can be changed after creation.

```
numbers = [10, 20, 30]
```

```
numbers[1] = 100
```

```
print(numbers)
```

Output:

```
[10, 100, 30]
```

Unlike strings, lists can be modified directly.

8.6 Adding Elements

append()

Adds one element to the **end** of a list.

```
numbers = [10, 20]
```

```
numbers.append(30)
```

```
print(numbers)
```

Output:

```
[10, 20, 30]
```

extend()

Adds multiple elements from an iterable.

```
numbers = [10, 20]
```

Prathamesh Arvind Jadhav

```
numbers.extend([30, 40])
```

```
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

append() vs extend()

```
numbers = [1, 2]
```

```
numbers.append([3, 4])
```

Result:

```
[1, 2, [3, 4]]
```

But:

```
numbers = [1, 2]
```

```
numbers.extend([3, 4])
```

Result:

```
[1, 2, 3, 4]
```

Remember:

append() → adds one object

extend() → adds elements from an iterable

insert()

Adds an element at a specific position.

```
numbers = [10, 20, 30]
```

```
numbers.insert(1, 15)
```

```
print(numbers)
```

Output:

[10, 15, 20, 30]

8.7 Removing Elements

remove()

Removes the **first matching value**.

```
numbers = [10, 20, 30, 20]
```

```
numbers.remove(20)
```

```
print(numbers)
```

Output:

```
[10, 30, 20]
```

If the value does not exist, `remove()` raises `ValueError`.

pop()

Removes and **returns** an element.

Without an index:

```
numbers = [10, 20, 30]
```

```
value = numbers.pop()
```

```
print(value)
```

```
print(numbers)
```

Output:

```
30
```

```
[10, 20]
```

With an index:

```
numbers = [10, 20, 30]
```

```
numbers.pop(1)
```

```
print(numbers)
```

Output:

```
[10, 30]
```

clear()

Removes all elements.

```
numbers = [10, 20, 30]
```

```
numbers.clear()
```

```
print(numbers)
```

Output:

```
[]
```

8.8 Updating Elements

Because lists are mutable, elements can be changed using their index.

```
numbers = [10, 20, 30]
```

```
numbers[1] = 200
```

```
print(numbers)
```

Output:

```
[10, 200, 30]
```

Multiple elements can be updated using slicing:

```
numbers[1:3] = [200, 300]
```

8.9 List Methods

append()

Prathamesh Arvind Jadhav

Adds one item at the end.

```
numbers.append(50)
```

extend()

Adds elements from another iterable.

```
numbers.extend([60, 70])
```

insert()

Adds an item at a specified index.

```
numbers.insert(1, 15)
```

remove()

Removes the first matching value.

```
numbers.remove(20)
```

pop()

Removes and returns an item.

```
numbers.pop()  
numbers.pop(0)
```

clear()

Removes all elements.

```
numbers.clear()
```

index()

Returns the index of the first matching value.

```
numbers = [10, 20, 30]
```

```
print(numbers.index(20))
```

Output:

```
1
```

count()

Counts how many times a value occurs.

```
numbers = [10, 20, 20, 30]
```

```
print(numbers.count(20))
```

Output:

```
2
```

sort()

Sorts the list in ascending order by default.

```
numbers = [40, 10, 30, 20]
```

```
numbers.sort()
```

```
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

Descending:

```
numbers.sort(reverse=True)
```

sort() modifies the original list and returns None.

reverse()

Reverses the order of elements.

Prathamesh Arvind Jadhav

```
numbers = [10, 20, 30]
```

```
numbers.reverse()
```

```
print(numbers)
```

Output:

```
[30, 20, 10]
```

reverse() modifies the original list and returns None.

copy()

Creates a **shallow copy** of a list.

```
numbers = [10, 20, 30]
```

```
new_numbers = numbers.copy()
```

Now numbers and new_numbers are separate list objects.

8.10 List Aliasing

Aliasing occurs when two variables refer to the **same list object**.

```
numbers = [10, 20, 30]
```

```
other = numbers
```

```
other.append(40)
```

```
print(numbers)
```

Output:

```
[10, 20, 30, 40]
```

Why?

```
numbers —┐  
          ↓
```

[10, 20, 30, 40]
↑
other _____

Both variables refer to the same list.

8.11 copy() vs Assignment

This is **very important for interviews**.

Assignment

```
a = [1, 2, 3]
b = a
```

```
b.append(4)
```

```
print(a)
```

Output:

```
[1, 2, 3, 4]
```

a and b refer to the same list.

copy()

```
a = [1, 2, 3]
b = a.copy()
```

```
b.append(4)
```

```
print(a)
```

```
print(b)
```

Output:

```
[1, 2, 3]
```

```
[1, 2, 3, 4]
```

They are separate outer list objects.

Remember

`b = a`
↓
Same object

`b = a.copy()`
↓
New shallow copy

8.12 Shallow Copy

A **shallow copy** creates a new outer list, but nested objects inside it may still be shared.

Example:

```
a = [[1, 2], [3, 4]]  
b = a.copy()
```

```
b[0].append(100)
```

```
print(a)
```

Output:

```
[[1, 2, 100], [3, 4]]
```

The nested list [1, 2] is shared.

For beginners, remember:

`copy()` → new outer list
→ nested objects may still be shared

A deep copy, using `copy.deepcopy()`, recursively copies nested objects.

Module 9: Tuples

A **tuple** is an ordered and immutable collection of elements. Tuples are useful when data should not be changed after creation.

9.1 Creating Tuples

Tuples are generally created using parentheses ().

```
numbers = (10, 20, 30)
```

A tuple can contain different data types:

```
data = (10, "Python", 3.14, True)
```

Single-element Tuple

A comma is required:

```
x = (10,)
```

Without the comma:

```
x = (10)
```

x is an integer, not a tuple.

9.2 Indexing

- Tuple indexing starts from 0.
- Negative indexing starts from -1.

```
numbers = (10, 20, 30, 40)
```

```
print(numbers[0]) # 10  
print(numbers[2]) # 30  
print(numbers[-1]) # 40
```

9.3 Slicing

Tuples support slicing.

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[1:4])
```

Output:

```
(20, 30, 40)
```

Other examples:

```
print(numbers[:3])    # (10, 20, 30)
print(numbers[2:])    # (30, 40, 50)
print(numbers[::-1])  # (50, 40, 30, 20, 10)
```

9.4 Tuple Packing

Tuple packing means putting multiple values into a single tuple.

```
student = "Prathamesh", 22, "AI & DS"
```

Python automatically creates:

```
("Prathamesh", 22, "AI & DS")
```

Explicitly:

```
student = ("Prathamesh", 22, "AI & DS")
```

9.5 Tuple Unpacking

Tuple unpacking means assigning tuple elements to multiple variables.

```
student = ("Prathamesh", 22, "AI & DS")
```

```
name, age, course = student
```

```
print(name)
print(age)
print(course)
```

Output:

```
Prathamesh
22
AI & DS
```

The number of variables should normally match the number of unpacked elements.

9.6 Nested Tuples

A tuple can contain other tuples.

```
data = (  
    (1, 2),  
    (3, 4),  
    (5, 6)  
)
```

Access an element:

```
print(data[0][1])
```

Output:

```
2
```

9.7 Tuple Immutability

- Tuples are **immutable**.
- Their elements cannot be changed, added, or removed after creation.

```
numbers = (10, 20, 30)
```

```
# numbers[0] = 100 # TypeError
```

However, a tuple can contain a mutable object such as a list:

```
data = ([1, 2], 3)
```

```
data[0].append(4)
```

```
print(data)
```

Output:

```
([1, 2, 4], 3)
```

The tuple itself is still immutable, but the nested list can change.

9.8 Tuple Methods

Tuples have relatively few methods because they are immutable.

count()

Counts occurrences of a value.

```
numbers = (10, 20, 20, 30)
```

```
print(numbers.count(20))
```

Output:

2

index()

Returns the index of the first occurrence.

```
numbers = (10, 20, 30)
```

```
print(numbers.index(20))
```

Output:

1

9.9 Tuple vs List

Tuple	List
Immutable	Mutable
()	[]
Generally used for fixed data	Used for changeable data
Fewer methods	More methods
Can be used as a dictionary key if all elements are hashable	Cannot be a dictionary key
Usually smaller/lighter for equivalent data	Generally more flexible

Remember

List → Mutable

Tuple → Immutable

Module 10: Sets

A **set** is a mutable collection of **unique, hashable elements**. Sets are useful for removing duplicates and performing mathematical set operations.

10.1 Creating Sets

Sets are created using `{ }` with elements.

```
numbers = {10, 20, 30}
```

Duplicate values are automatically removed:

```
numbers = {10, 20, 20, 30}
```

```
print(numbers)
```

The set contains only unique elements.

Empty Set

Important:

```
s = set()
```

Do **not** use:

```
s = { }
```

because `{ }` creates an empty dictionary.

10.2 Set Properties

Important properties of sets:

- Store **unique elements**.
- Mutable.
- Elements must be **hashable**.
- Do not support indexing like lists.
- Set iteration order should not be relied upon.
- Useful for membership testing and set operations.

Example:

```
numbers = {10, 20, 30}
```

```
print(20 in numbers)
```


Output:

True

10.3 Adding Elements

Use `add()` to add one element.

```
numbers = {10, 20}
```

```
numbers.add(30)
```

```
print(numbers)
```

10.4 Removing Elements

Sets provide several ways to remove elements.

`remove()`

Removes an element.

```
numbers = {10, 20, 30}
```

```
numbers.remove(20)
```

If the element does not exist, `remove()` raises `KeyError`.

`discard()`

Removes an element if present.

```
numbers = {10, 20, 30}
```

```
numbers.discard(50)
```

No error occurs if 50 is not present.

`pop()`

Removes and returns an **arbitrary** element.

```
numbers = {10, 20, 30}
```

```
value = numbers.pop()
```

```
print(value)
```

Do not depend on which element pop() removes.

10.5 Set Operations

Suppose:

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

Union

- Combines all unique elements from both sets.
- Operator: |
- Method: .union()

```
print(A | B)
```

Result:

{1, 2, 3, 4, 5, 6}

```
print(A.union(B))
```

Intersection

- Returns elements common to both sets.
- Operator: &
- Method: .intersection()

```
print(A & B)
```

Result:

{3, 4}

Difference

- Returns elements present in the first set but not the second.

- Operator: -

```
print(A - B)
```

Result:

```
{1, 2}
```

```
print(B - A)
```

Result:

```
{5, 6}
```

Symmetric Difference

- Returns elements that are in either set, but **not in both**.
- Operator: ^

```
print(A ^ B)
```

Result:

```
{1, 2, 5, 6}
```

10.6 Subset

A set is a **subset** if all its elements are present in another set.

```
A = {1, 2}
```

```
B = {1, 2, 3, 4}
```

```
print(A.issubset(B))
```

Output:

```
True
```

Operator:

```
print(A <= B)
```

10.7 Superset

A set is a **superset** if it contains all elements of another set.

A = {1, 2}

B = {1, 2, 3, 4}

```
print(B.issuperset(A))
```

Output:

True

Operator:

```
print(B >= A)
```

10.8 Disjoint Sets

Two sets are **disjoint** if they have no common elements.

A = {1, 2}

B = {3, 4}

```
print(A.isdisjoint(B))
```

Output:

True

10.9 Set Methods

add()

Adds one element.

```
numbers.add(40)
```

remove()

Removes an element; raises KeyError if absent.

```
numbers.remove(20)
```

discard()

Prathamesh Arvind Jadhav

Removes an element if present; no error if absent.

```
numbers.discard(50)
```

pop()

Removes and returns an arbitrary element.

```
numbers.pop()
```

update()

Adds elements from another iterable.

```
numbers = {1, 2}
```

```
numbers.update([3, 4, 5])
```

```
print(numbers)
```

Result:

```
{1, 2, 3, 4, 5}
```

Module 11: Dictionaries

A **dictionary** stores data as **key-value pairs**. It is one of the most important Python data structures and is heavily used in real-world applications and coding interviews.

11.1 Dictionary Basics

A dictionary is written using {} and stores data in the form:

key → value

Example:

```
student = {  
    "name": "Prathamesh",  
    "age": 22,  
    "course": "AI & DS"  
}
```

Here:

"name" → key

"Prathamesh" → value

11.2 Key-Value Pairs

Each dictionary entry contains:

key : value

Example:

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

- "name" and "age" → keys
- "Prathamesh" and 22 → values

Keys must be **unique** within a dictionary.

If the same key is assigned again, the newer value replaces the previous one.

```
data = {  
    "name": "Prathamesh",  
    "name": "Rahul"  
}
```

```
print(data)
```

Result:

```
{'name': 'Rahul'}
```

11.3 Creating Dictionaries

Normal Dictionary

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

Empty Dictionary

```
student = {}
```

or:

```
student = dict()
```

Dictionary with Different Value Types

```
data = {  
    "name": "Python",  
    "version": 3,  
    "price": 10.5,  
    "available": True  
}
```

11.4 Accessing Values

Use the key inside square brackets.

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

```
print(student["name"])
```

Output:

Prathamesh

If the key doesn't exist, [] raises KeyError.

Using get()

get() safely retrieves a value.

```
print(student.get("name"))
```

If the key does not exist:

```
print(student.get("city"))
```

Prathamesh Arvind Jadhav

Result:

None

You can provide a default:

```
print(student.get("city", "Not Available"))
```

11.5 Adding and Updating Values

Add a New Key

```
student["city"] = "Pune"
```

Update Existing Key

```
student["age"] = 23
```

Example:

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

```
student["age"] = 23  
student["city"] = "Pune"
```

```
print(student)
```

11.6 Removing Values

pop()

Removes a specific key and returns its value.

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

```
age = student.pop("age")
```

```
print(age)
```



```
print(student)
```

popitem()

Removes and returns the **last inserted key-value pair**.

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

```
item = student.popitem()
```

```
print(item)
```

clear()

Removes all dictionary items.

```
student.clear()
```

```
print(student)
```

Output:

```
{}
```

11.7 Nested Dictionaries

A dictionary can contain another dictionary.

```
students = {  
    "student1": {  
        "name": "Prathamesh",  
        "age": 22  
    },  
    "student2": {  
        "name": "Rahul",  
        "age": 23  
    }  
}
```

Access nested values:

Prathamesh Arvind Jadhav

```
print(students["student1"]["name"])
```

Output:

Prathamesh

Nested dictionaries are common when representing structured data such as JSON.

11.8 Dictionary Methods

keys()

Returns a view containing dictionary keys.

```
student = {  
    "name": "Prathamesh",  
    "age": 22  
}
```

```
print(student.keys())
```

values()

Returns a view containing dictionary values.

```
print(student.values())
```

items()

Returns key-value pairs.

```
print(student.items())
```

Very commonly used with loops:

```
for key, value in student.items():  
    print(key, value)
```

get()

Returns the value associated with a key.

```
print(student.get("name"))
```

update()

Adds or updates multiple key-value pairs.

```
student.update({  
    "age": 23,  
    "city": "Pune"  
})
```

pop()

Removes a specified key.

```
student.pop("city")
```

popitem()

Removes and returns the last inserted item.

```
student.popitem()
```

setdefault()

Returns the value for a key.

If the key does not exist, it inserts the key with the specified default value.

```
student = {  
    "name": "Prathamesh"  
}
```

```
student.setdefault("age", 22)
```

```
print(student)
```

Result:

```
{'name': 'Prathamesh', 'age': 22}
```

If the key already exists, its value is not replaced.

Prathamesh Arvind Jadhav

```
student.setdefault("name", "Rahul")  
print(student["name"])
```

Output:

Prathamesh

clear()

Removes all items.

```
student.clear()
```

11.9 Dictionary Keys

Dictionary keys must be **hashable**.

Valid keys:

```
data = {  
    "name": "Python",  
    1: "One",  
    (1, 2): "Tuple"  
}
```

Invalid key:

```
# data = {[1, 2]: "List"} # TypeError
```

A list cannot be a dictionary key because lists are mutable and unhashable.

11.10 Hashability

A **hashable object** can be used as:

- A dictionary key
- A set element

Common hashable types:

int

float
str
bool
tuple (if its elements are hashable)

Common unhashable types:

list
set
dict

Simple rule

Dictionary key → Must be hashable
Set element → Must be hashable

11.11 Dictionary Comprehension

Dictionary comprehension provides a short way to create dictionaries.

Normal approach

```
squares = {}  
  
for i in range(1, 6):  
    squares[i] = i * i  
  
print(squares)
```

Dictionary comprehension

```
squares = {i: i * i for i in range(1, 6)}  
  
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

With condition

```
even_squares = {  
    i: i * i  
    for i in range(1, 6)  
    if i % 2 == 0  
}
```

Prathamesh Arvind Jadhav

```
}
```

```
print(even_squares)
```

Output:

```
{2: 4, 4: 16}
```

11.12 Nested Dictionaries

Nested dictionaries are dictionaries inside dictionaries.

Example:

```
company = {  
    "employee1": {  
        "name": "Prathamesh",  
        "role": "Developer",  
        "skills": ["Python", "SQL"]  
    },  
    "employee2": {  
        "name": "Rahul",  
        "role": "Tester",  
        "skills": ["Python", "Selenium"]  
    }  
}
```

Accessing nested data:

```
print(company["employee1"]["role"])
```

Output:

Developer

Accessing nested list:

```
print(company["employee1"]["skills"][0])
```

Output:

Python

Module 12: Functions

A **function** is a reusable block of code that performs a specific task. Functions help make programs **modular, reusable, readable, and easier to maintain**.

12.1 What is a Function?

A function is a named block of code that runs when it is called.

```
def greet():  
    print("Hello Python")  
  
greet()
```

Output:

Hello Python

Why use functions?

- Avoid repeating code.
 - Make programs easier to understand.
 - Reuse the same logic multiple times.
 - Make testing and debugging easier.
-

12.2 Defining Functions

Use the `def` keyword to define a function.

Syntax

```
def function_name():  
    # function body
```

Example:

```
def greet():  
    print("Hello")  
  
greet()
```

12.3 Calling Functions

Calling a function means executing it.

```
def greet():  
    print("Hello")
```

```
greet()  
greet()
```

Output:

```
Hello  
Hello
```

A function can be called multiple times.

12.4 Parameters

A **parameter** is a variable defined inside the function definition that receives a value.

```
def greet(name):  
    print("Hello", name)
```

Here, name is a parameter.

12.5 Arguments

An **argument** is the actual value passed to a function when it is called.

```
greet("Prathamesh")
```

Here:

```
name      → Parameter  
"Prathamesh" → Argument
```

12.6 Return Values

The return statement sends a value back to the caller.


```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
  
print(result)
```

Output:

30

return vs print()

```
def add(a, b):  
    return a + b
```

return gives the value back so it can be stored or used in another expression.

```
result = add(10, 20)
```

print() only displays a value.

12.7 Local Variables

A variable created inside a function is generally **local to that function**.

```
def calculate():  
    x = 10  
    print(x)
```

```
calculate()
```

x cannot normally be accessed outside the function.

```
def calculate():  
    x = 10
```

```
calculate()
```

```
# print(x) # NameError
```

12.8 Global Variables

A variable defined outside functions is generally a **global variable**.

```
x = 100
```

```
def display():  
    print(x)
```

```
display()
```

The function can read the global variable.

12.9 Types of Arguments

1. Positional Arguments

Arguments are matched according to their position.

```
def student(name, age):  
    print(name, age)
```

```
student("Prathamesh", 22)
```

Here:

```
name → "Prathamesh"  
age  → 22
```

2. Keyword Arguments

Arguments are passed using parameter names.

```
def student(name, age):  
    print(name, age)
```

```
student(age=22, name="Prathamesh")
```

The order does not matter when using parameter names.

3. Default Arguments

A parameter can have a default value.

Prathamesh Arvind Jadhav

```
def greet(name="User"):
    print("Hello", name)
```

```
greet()
greet("Prathamesh")
```

Output:

```
Hello User
Hello Prathamesh
```

If an argument is provided, it replaces the default value.

12.10 Variable-Length Arguments

Used when a function needs to accept a variable number of arguments.

***args**

- *args collects multiple **positional arguments** into a tuple.

```
def add(*args):
    print(args)
```

```
add(10, 20, 30)
```

Output:

```
(10, 20, 30)
```

Example:

```
def add(*args):
    return sum(args)
```

```
print(add(10, 20, 30))
```

Output:

```
60
```

Remember

*args → multiple positional arguments
→ stored as tuple

****kwargs**

- **kwargs collects multiple **keyword arguments** into a dictionary.

```
def student(**kwargs):  
    print(kwargs)
```

```
student(name="Prathamesh", age=22)
```

Output:

```
{'name': 'Prathamesh', 'age': 22}
```

Remember

**kwargs → multiple keyword arguments
→ stored as dictionary

12.11 Function Scope

Scope defines where a variable can be accessed.

Common scopes in Python:

Local
Enclosing
Global
Built-in

This is often remembered as **LEGB**:

L → Local
E → Enclosing
G → Global
B → Built-in

Example:

```
x = "global"
```

```
def outer():
```

```
x = "enclosing"
```

```
def inner():  
    x = "local"  
    print(x)
```

```
inner()
```

```
outer()
```

Output:

```
local
```

12.12 global

The global keyword allows a function to assign to a global variable.

```
count = 0
```

```
def increase():  
    global count  
    count += 1
```

```
increase()
```

```
print(count)
```

Output:

```
1
```

Without global, assigning to count inside the function would normally create a local variable instead.

Use global carefully; excessive use can make programs harder to maintain.

12.13 nonlocal

The nonlocal keyword allows an inner function to modify a variable from its **enclosing function**.

```
def outer():  
    count = 0
```

```
def inner():  
    nonlocal count  
    count += 1
```

```
inner()  
print(count)
```

```
outer()
```

Output:

1

Difference

global → variable from global scope

nonlocal → variable from enclosing function scope

12.14 First-Class Functions

Python treats functions as **first-class objects**.

This means functions can be:

- Stored in variables.
- Passed as arguments.
- Returned from other functions.
- Stored in collections.

Example:

```
def greet():  
    print("Hello")
```

```
message = greet
```

```
message()
```

Output:

Hello

Here, message refers to the same function object as greet.

12.15 Functions as Objects

Functions can be assigned to variables.

```
def square(x):  
    return x * x
```

```
f = square
```

```
print(f(5))
```

Output:

25

The function can be treated like other Python objects.

12.16 Passing Functions as Arguments

A function can be passed to another function.

```
def square(x):  
    return x * x
```

```
def calculate(func, value):  
    return func(value)
```

```
print(calculate(square, 5))
```

Output:

25

Here, square is passed as an argument.

12.17 Returning Functions

A function can return another function.

```
def outer():
```

```
def inner():  
    print("Hello")  
  
    return inner  
  
func = outer()  
  
func()
```

Output:

Hello

This concept is important for understanding **closures and decorators** later.

Module 13: Lambda Functions

A **lambda function** is a small anonymous function, usually used when a simple function is needed temporarily.

13.1 Lambda Function

A lambda function is a function without a regular def name.

Example:

```
square = lambda x: x * x  
  
print(square(5))
```

Output:

25

13.2 Lambda Syntax

Syntax

lambda arguments: expression

Example:

Prathamesh Arvind Jadhav

```
add = lambda a, b: a + b
```

```
print(add(10, 20))
```

Output:

30

Important:

- Lambda can have multiple arguments.
- Lambda contains **one expression**.
- The expression's result is returned automatically.

13.3 Lambda with Arguments

One argument

```
square = lambda x: x * x
```

```
print(square(4))
```

Multiple arguments

```
add = lambda a, b: a + b
```

```
print(add(10, 20))
```

With condition

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
```

```
print(check(10))
```

Output:

Even

13.4 Lambda with map()

map() applies a function to every element of an iterable.

Example:

Prathamesh Arvind Jadhav

```
numbers = [1, 2, 3, 4]
squares = list(map(lambda x: x * x, numbers))
print(squares)
```

Output:

```
[1, 4, 9, 16]
```

Remember

map() → transform each element

13.5 Lambda with filter()

filter() selects elements for which a condition is true.

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
even = list(filter(lambda x: x % 2 == 0, numbers))
print(even)
```

Output:

```
[2, 4, 6]
```

Remember

filter() → select elements

13.6 Lambda with sorted()

sorted() can use a key function to determine sorting criteria.

Example:

```
students = [
    ("Prathamesh", 85),
    ("Rahul", 70),
    ("Amit", 90)
```

```
]
```

```
result = sorted(students, key=lambda student: student[1])
```

```
print(result)
```

Output:

```
[('Rahul', 70), ('Prathamesh', 85), ('Amit', 90)]
```

Here:

```
lambda student: student[1]
```

tells sorted() to sort using the marks.

Descending order:

```
result = sorted(  
    students,  
    key=lambda student: student[1],  
    reverse=True  
)
```

13.7 Lambda vs Normal Function

Lambda	Normal Function
Anonymous function	Named function
Uses lambda	Uses def
Usually one expression	Can contain multiple statements
Good for short operations	Better for complex logic
Usually used temporarily	Easily reusable and documented

Lambda

```
square = lambda x: x * x
```

Normal function

```
def square(x):  
    return x * x
```

Both produce the same result.

Module 14: Comprehensions

Comprehensions provide a short and readable way to create collections from existing iterables.

They are very common in Python coding and interviews.

14.1 List Comprehension

List comprehension is used to create a new list in a single line.

Basic Syntax

[expression for item in iterable]

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = [x * x for x in numbers]
```

```
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

Equivalent Normal Loop

```
squares = []
```

```
for x in numbers:  
    squares.append(x * x)
```

So:

List comprehension → Shorter way to create a list

14.2 Conditional Comprehension

A condition can be added using if.

Syntax

[expression for item in iterable if condition]

Example:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even = [x for x in numbers if x % 2 == 0]
```

```
print(even)
```

Output:

```
[2, 4, 6]
```

With transformation

```
squares = [x * x for x in numbers if x % 2 == 0]
```

```
print(squares)
```

Output:

```
[4, 16, 36]
```

14.3 Conditional Expression in Comprehension

You can use if-else inside the expression.

Syntax

[expression_if_true if condition else expression_if_false for item in iterable]

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
result = ["Even" if x % 2 == 0 else "Odd" for x in numbers]
```

```
print(result)
```

Output:

```
['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

Important Difference

```
[x for x in numbers if x % 2 == 0]
```

Here if **filters** elements.

```
["Even" if x % 2 == 0 else "Odd" for x in numbers]
```

Here if-else **chooses the output value**.

14.4 Nested List Comprehension

A comprehension can contain multiple for clauses.

Example:

```
result = [  
    (x, y)  
    for x in range(2)  
    for y in range(3)  
]
```

```
print(result)
```

Output:

```
[(0, 0), (0, 1), (0, 2),  
 (1, 0), (1, 1), (1, 2)]
```

Nested List Example

Flatten a nested list:

```
matrix = [  
    [1, 2],  
    [3, 4],  
    [5, 6]  
]
```

```
result = [x for row in matrix for x in row]
```

```
print(result)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

14.5 Dictionary Comprehension

Dictionary comprehension creates dictionaries concisely.

Syntax

```
{key_expression: value_expression for item in iterable}
```

Example:

```
squares = {x: x * x for x in range(1, 6)}  
  
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

With Condition

```
even_squares = {  
    x: x * x  
    for x in range(1, 6)  
    if x % 2 == 0  
}
```

```
print(even_squares)
```

Output:

```
{2: 4, 4: 16}
```

14.6 Set Comprehension

Set comprehension creates a set.

Syntax

```
{expression for item in iterable}
```

Example:

```
numbers = [1, 2, 2, 3, 3, 4]
```

```
squares = {x * x for x in numbers}
```

```
print(squares)
```

Output:

```
{1, 4, 9, 16}
```

Duplicate results are automatically removed because sets store unique elements.

14.7 Generator Expressions

A **generator expression** looks similar to a list comprehension but uses `()`.

```
numbers = (x * x for x in range(5))
```

It produces values **lazily**, rather than creating the entire collection immediately.

Example:

```
numbers = (x * x for x in range(5))
```

```
for value in numbers:  
    print(value)
```

Output:

```
0  
1  
4  
9  
16
```

List vs Generator

```
[x * x for x in range(1000)]
```

Creates the entire list in memory.

```
(x * x for x in range(1000))
```

Generates values as needed.

Remember

[] → List comprehension
{ } → Dictionary/Set comprehension
() → Generator expression

Module 15: Built-in Functions

Built-in functions are functions provided by Python that can be used directly without importing a module.

15.1 len()

Returns the number of elements.

```
numbers = [10, 20, 30]
```

```
print(len(numbers))
```

Output:

3

Works with strings, lists, tuples, dictionaries, etc.

15.2 type()

Returns the type of an object.

```
x = 10
```

```
print(type(x))
```

Output:

```
<class 'int'>
```

15.3 id()

Returns the identity of an object during its lifetime.

```
x = 10
```

```
print(id(x))
```

The exact number can vary between executions.

15.4 range()

Generates a sequence of numbers, commonly used in loops.

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

15.5 sum()

Returns the sum of elements.

```
numbers = [10, 20, 30]  
  
print(sum(numbers))
```

Output:

```
60
```

15.6 min()

Returns the smallest value.

```
numbers = [10, 5, 20]  
  
print(min(numbers))
```

Output:

5

15.7 max()

Returns the largest value.

```
numbers = [10, 5, 20]
```

```
print(max(numbers))
```

Output:

20

15.8 abs()

Returns the absolute value.

```
print(abs(-10))
```

Output:

10

15.9 round()

Rounds a number.

```
print(round(3.14159))
```

Output:

3

You can specify decimal places:

```
print(round(3.14159, 2))
```

Output:

3.14

15.10 sorted()

Returns a **new sorted list**.

```
numbers = [30, 10, 20]
result = sorted(numbers)
print(result)
```

Output:

```
[10, 20, 30]
```

The original list is not modified.

sorted() vs list.sort()

sorted() → returns a new sorted list
sort() → modifies the original list

15.11 reversed()

Returns an iterator that produces elements in reverse order.

```
numbers = [1, 2, 3, 4]
result = reversed(numbers)
print(list(result))
```

Output:

```
[4, 3, 2, 1]
```

15.12 enumerate()

Adds a counter while iterating.

```
names = ["Amit", "Rahul", "Prathamesh"]

for index, name in enumerate(names):
    print(index, name)
```

Output:

```
0 Amit
1 Rahul
2 Prathamesh
```

You can specify the starting index:

```
for index, name in enumerate(names, start=1):
    print(index, name)
```

15.13 zip()

Combines elements from multiple iterables.

```
names = ["Amit", "Rahul"]
ages = [22, 23]
```

```
result = zip(names, ages)
```

```
print(list(result))
```

Output:

```
[('Amit', 22), ('Rahul', 23)]
```

Commonly used for parallel iteration:

```
for name, age in zip(names, ages):
    print(name, age)
```

15.14 map()

Applies a function to every element.

```
numbers = [1, 2, 3, 4]
```

```
result = map(lambda x: x * 2, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4, 6, 8]
```

Remember:

`map()` → transform elements

15.15 filter()

Selects elements based on a condition.

```
numbers = [1, 2, 3, 4, 5]
```

```
result = filter(lambda x: x % 2 == 0, numbers)
```

```
print(list(result))
```

Output:

```
[2, 4]
```

Remember:

`filter()` → select elements

15.16 any()

Returns True if **at least one** element is truthy.

```
values = [False, False, True]
```

```
print(any(values))
```

Output:

```
True
```

Example:

```
numbers = [1, 3, 5, 8]
```

```
print(any(x % 2 == 0 for x in numbers))
```

Output:

```
True
```

15.17 all()

Returns True if **all** elements are truthy.

```
values = [True, True, True]
```

```
print(all(values))
```

Output:

True

Example:

```
numbers = [2, 4, 6, 8]
```

```
print(all(x % 2 == 0 for x in numbers))
```

Output:

True

any() vs all()

any() → At least one is True

all() → Every element is True

Module 16: Python Modules

A **module** is a Python file containing reusable code such as functions, classes, and variables.

Modules help organize code and allow you to reuse functionality.

16.1 What is a Module?

A Python file with a .py extension can be used as a module.

Example:

calculator.py

It can contain:

```
def add(a, b):
```

```
return a + b
```

Another Python file can import and use it.

16.2 Creating Modules

Create a file called:

calculator.py

Add:

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

Now this file can be imported into another Python program.

16.3 Importing Modules

Use import.

```
import calculator
```

```
print(calculator.add(10, 20))
```

Output:

30

Access members using:

```
module_name.member
```

16.4 import

Imports the complete module namespace.

```
import math
```



```
print(math.sqrt(25))
```

Output:

```
5.0
```

16.5 from ... import

Imports specific members from a module.

```
from math import sqrt
```

```
print(sqrt(25))
```

You can import multiple members:

```
from math import sqrt, factorial
```

```
print(sqrt(25))
```

```
print(factorial(5))
```

16.6 as

as creates an alias for a module or imported member.

```
import math as m
```

```
print(m.sqrt(25))
```

Another example:

```
from math import factorial as fact
```

```
print(fact(5))
```

16.7 Multiple Imports

You can import multiple modules.

```
import math
```

```
import random
```

```
import datetime
```

Or:

```
import math, random
```

For readability, separate import statements are generally preferred.

16.8 `__name__`

`__name__` is a special built-in variable that tells you the name under which a module is being executed.

If a Python file is run directly:

```
print(__name__)
```

It usually outputs:

```
__main__
```

If the file is imported, `__name__` contains the module's name.

16.9 `__main__`

When a Python file is executed directly, Python sets:

```
__name__ == "__main__"
```

This allows you to separate reusable code from code that should run only when the file is executed directly.

Example:

```
def greet():  
    print("Hello")  
  
if __name__ == "__main__":  
    greet()
```

Why use this?

If the file is:

python_file.py

and executed directly:

```
python python_file.py
```

the greet() call runs.

If the file is imported by another module, the greet() call inside the if block does not automatically run.

Remember

```
__name__ == "__main__"
```

↓

File is being executed directly

16.10 Important Module: math

The math module provides mathematical functions and constants.

```
import math
```

```
print(math.sqrt(25))  
print(math.factorial(5))  
print(math.pi)
```

Common functions:

```
sqrt()  
factorial()  
ceil()  
floor()  
pow()
```

16.11 random

The random module is used to generate pseudo-random values.

```
import random
```

```
print(random.randint(1, 10))
```

Common functions:

```
random()
randint()
choice()
shuffle()
```

Example:

```
items = ["Python", "Java", "C++"]

print(random.choice(items))
```

16.12 datetime

The datetime module is used for dates and times.

```
from datetime import datetime

now = datetime.now()

print(now)
```

You can access individual parts:

```
print(now.year)
print(now.month)
print(now.day)
```

16.13 os

The os module provides functions for interacting with the operating system.

```
import os

print(os.getcwd())
```

Common uses:

- Current working directory
- File and directory operations
- Environment variables
- Path-related operations

Example:

```
print(os.listdir())
```

16.14 sys

The sys module provides access to Python runtime/system information.

```
import sys
```

```
print(sys.version)
```

It can also access command-line arguments:

```
print(sys.argv)
```

Common uses:

- Python version information
 - Command-line arguments
 - Interpreter-related information
-

16.15 statistics

The statistics module provides common statistical calculations.

```
import statistics
```

```
numbers = [10, 20, 30, 40, 50]
```

```
print(statistics.mean(numbers))
```

```
print(statistics.median(numbers))
```

Common functions:

```
mean()
```

```
median()
```

```
mode()
```

```
stdev()
```

16.16 json

The json module is used to work with **JSON data**.

JSON is commonly used for APIs and data exchange.

Python Dictionary → JSON

```
import json

data = {
    "name": "Prathamesh",
    "age": 22
}

json_data = json.dumps(data)

print(json_data)
```

JSON → Python Object

```
python_data = json.loads(json_data)

print(python_data)
```

Important:

json.dumps() → Python object → JSON string
json.loads() → JSON string → Python object

Module 17: Packages

A package is a way to organize related Python modules into folders. Packages help manage large projects by grouping modules logically.

17.1 What is a Package?

- A module is a single .py file.
- A package is a directory containing related Python modules.

Example:

```
myproject/
├── main.py
├── utilities/
│   ├── __init__.py
│   ├── math_utils.py
│   └── string_utils.py
```

Here:

- `math_utils.py` → module
- `string_utils.py` → module
- `utilities` → package

Remember

Module → One Python file

Package → Folder containing related modules

17.2 Package Structure

Example:

```
shop/
├── __init__.py
├── products.py
├── orders.py
└── payments/
    ├── __init__.py
    └── gateway.py
```

This package contains:

- Modules: `products.py`, `orders.py`
- Nested package: `payments`

17.3 `__init__.py`

`__init__.py` is a special file inside a package.

Historically, it marked a directory as a Python package. In modern Python, namespace packages can exist without it, but it is still commonly used to:

- initialize package-level code,
- expose selected names,
- and control package imports.

Example:

```
# utilities/__init__.py
```

```
print("Utilities package loaded")
```

When the package is imported, this code runs.

It can also expose functions:

```
from .math_utils import add
```

Then users can access:

```
from utilities import add
```

17.4 Importing Packages

Suppose:

```
utilities/  
├── __init__.py  
└── math_utils.py
```

math_utils.py:

```
def add(a, b):  
    return a + b
```

Import the module:

```
import utilities.math_utils  
  
print(utilities.math_utils.add(10, 20))
```

Or:

```
from utilities.math_utils import add  
  
print(add(10, 20))
```

17.5 Nested Packages

Packages can contain other packages.

Example:

```
project/  
├── database/  
│   └── __init__.py
```



```
└─ mysql/  
    └─ __init__.py  
    └─ connection.py
```

Import:

```
from database.mysql.connection import connect
```

Nested packages help organize larger applications.

17.6 Absolute Imports

An absolute import starts from the top-level package.

Example structure:

```
project/  
├── main.py  
└── app/  
    ├── __init__.py  
    ├── models.py  
    └── services.py
```

Inside services.py:

```
from app.models import User
```

This is an absolute import.

Advantage

- Clear and easy to understand.
- Usually preferred in larger projects.

17.7 Relative Imports

A relative import uses dots (.) to refer to the current package.

Inside services.py:

```
from .models import User
```

The dot means:

Current package

Examples:

. → current package
.. → parent package
... → grandparent package

Example:

```
from ..config import settings
```

This imports settings from the parent package.

Absolute vs Relative Imports

Absolute	Relative
Starts from top package	Starts from current package
Easier to understand	Shorter inside packages
Common in large projects	Useful within package modules

Module 18: Exception Handling

Exception handling allows a program to handle runtime errors gracefully instead of terminating unexpectedly.

18.1 Errors vs Exceptions

Errors

Errors are problems in a program.

Two common categories:

Syntax Errors
Runtime Errors (Exceptions)

Exceptions

Exceptions are errors that occur while the program is running.

Example:

```
print(10 / 0)
```

This raises:

ZeroDivisionError

18.2 Syntax Errors

Syntax errors occur when Python code does not follow Python's grammar.

Example:

```
if True
    print("Hello")
```

Missing : causes a SyntaxError.

The program cannot start until the syntax is corrected.

18.3 Runtime Errors

Runtime errors occur while the program is executing.

Example:

```
number = int("hello")
```

The syntax is valid, but it raises:

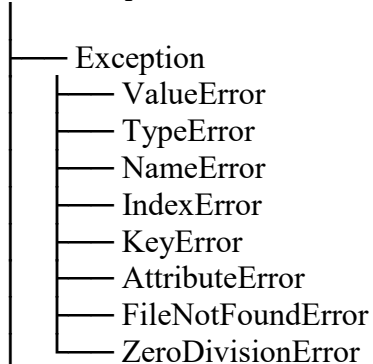
ValueError

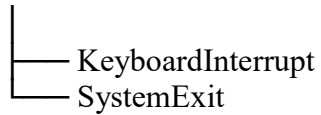
18.4 Exception Hierarchy

Python exceptions are organized in a hierarchy.

Simplified:

BaseException





Most application errors inherit from Exception.

18.5 try

Code that might raise an exception is placed inside try.

```
try:
    result = 10 / 0
```

18.6 except

except handles an exception.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output:

Cannot divide by zero

The program continues running.

18.7 else

The else block executes only if no exception occurs.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error")
else:
    print("Result:", result)
```

Output:

Result: 5.0

18.8 finally

The finally block always executes, whether an exception occurs or not.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error")
finally:
    print("Execution completed")
```

Output:

Error
Execution completed

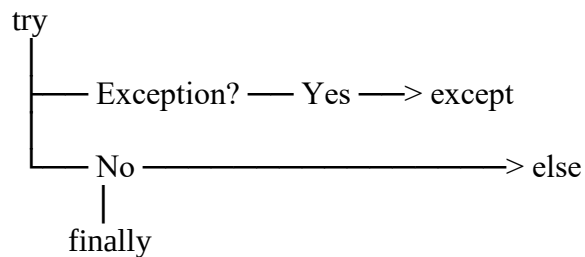
Common uses:

- Closing files
- Closing database connections
- Releasing resources

18.9 Complete Structure

```
try:
    # risky code
except SomeException:
    # handle error
else:
    # runs if no exception
finally:
    # always runs
```

Execution flow:



18.10 Multiple Exceptions

Different exceptions can be handled separately.

```
try:
    value = int(input("Enter number: "))
    result = 10 / value
```

```
except ValueError:  
    print("Invalid number")
```

```
except ZeroDivisionError:  
    print("Cannot divide by zero")
```

You can also handle multiple exceptions together:

```
except (ValueError, TypeError):  
    print("Invalid input")
```

18.11 Nested Exception Handling

A try-except block can be placed inside another one.

```
try:  
    try:  
        result = 10 / 0  
    except ZeroDivisionError:  
        print("Inner exception")
```

```
except Exception:  
    print("Outer exception")
```

Nested exception handling is useful when different layers of code need separate handling.

18.12 Raising Exceptions

Sometimes you want to create an exception intentionally.

Use raise.

```
age = -5
```

```
if age < 0:  
    raise ValueError("Age cannot be negative")
```

This stops execution and raises a ValueError.

18.13 raise

Syntax:

```
raise ExceptionType("message")
```

Example:

```
marks = 120
```

```
if marks > 100:  
    raise ValueError("Marks cannot exceed 100")
```

18.14 Custom Exceptions

You can create your own exception classes.

Custom exceptions normally inherit from Exception.

```
class InvalidAgeError(Exception):  
    pass
```

Use it:

```
age = -1  
  
if age < 0:  
    raise InvalidAgeError("Invalid age")
```

Custom exceptions make application-specific errors easier to understand.

18.15 Common Exceptions

Exception	Meaning
ValueError	Correct type, invalid value
TypeError	Wrong type for an operation
NameError	Variable not defined
IndexError	Invalid list/tuple index
KeyError	Dictionary key not found
ZeroDivisionError	Division by zero
FileNotFoundError	File does not exist
AttributeError	Object does not have the requested attribute

Examples

ValueError

```
int("hello")
```

TypeError

```
"hello" + 10
```

NameError

```
print(x)
```

IndexError

```
numbers = [1, 2]
```

```
print(numbers[5])
```

KeyError

```
data = {"name": "Python"}
```

```
print(data["age"])
```

ZeroDivisionError

```
10 / 0
```

FileNotFoundError

```
open("missing.txt")
```

AttributeError

```
x = 10
```

```
x.append(20)
```

Module 19: File Handling

File handling allows Python programs to read data from files and write data to files.

19.1 Opening Files

Use the `open()` function.

Syntax:

```
open(filename, mode)
```


Example:

```
file = open("data.txt", "r")
```

- "data.txt" → file name
- "r" → read mode

19.2 Reading Files

read()

Reads the entire file.

```
file = open("data.txt", "r")
```

```
content = file.read()
```

```
print(content)
```

```
file.close()
```

readline()

Reads one line at a time.

```
file = open("data.txt", "r")
```

```
line = file.readline()
```

```
print(line)
```

```
file.close()
```

Calling readline() again reads the next line.

readlines()

Reads all lines into a list.

```
file = open("data.txt", "r")
```

```
lines = file.readlines()
```

```
print(lines)
```

```
file.close()
```

Example output:

```
['Line 1\n', 'Line 2\n']
```

19.3 Writing Files

Use "w" mode.

```
file = open("data.txt", "w")
```

```
file.write("Hello Python")
```

```
file.close()
```

Important

"w" mode:

- Creates the file if it does not exist.
- Overwrites existing content if the file exists.

19.4 Appending Files

Use "a" mode.

```
file = open("data.txt", "a")
```

```
file.write("\nNew line")
```

```
file.close()
```

Append mode adds data to the end without deleting existing content.

19.5 Closing Files

Always close files after using them.

```
file.close()
```

Closing a file:

- Releases system resources.
- Ensures written data is flushed to disk.

19.6 File Modes

Mode	Meaning
r	Read
w	Write (overwrite)
a	Append
x	Create new file
rb	Read binary
wb	Write binary

r

- File must exist.
- Read only.

w

- Creates file if needed.
- Deletes old content.

a

- Creates file if needed.
- Adds data at the end.

x

- Creates a new file.
- Raises `FileExistsError` if it already exists.

rb

Reads binary files such as images or PDFs.

wb

Writes binary data.

19.7 read()

```
with open("data.txt", "r") as file:  
    content = file.read()
```

```
print(content)
```

Reads the entire file as one string.

19.8 readline()

```
with open("data.txt", "r") as file:  
    line = file.readline()
```

```
print(line)
```

Reads one line.

19.9 readlines()

```
with open("data.txt", "r") as file:
```

Module 20: Object-Oriented Programming — OOP

OOP (Object-Oriented Programming) is a programming approach where programs are designed using **classes and objects**.

OOP helps organize large programs and makes code reusable and maintainable.

20.1 Class

A **class** is a blueprint/template for creating objects.

```
class Student:  
    pass
```

A class can contain:

- Attributes → data
 - Methods → functions/behavior
-

20.2 Object

An **object** is an instance of a class.

```
class Student:  
    pass
```

```
student1 = Student()
```

Here:

Student → Class
student1 → Object

20.3 Attributes

Attributes are variables associated with an object or class.

```
class Student:
    name = "Prathamesh"
    age = 22
```

Here name and age are attributes.

20.4 Methods

A **method** is a function defined inside a class.

```
class Student:

    def greet(self):
        print("Hello")
```

20.5 Constructor

A constructor is used to initialize an object's attributes.

In Python, the commonly used constructor method is:

```
__init__()
```

Example:

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Creating object:

```
student = Student("Prathamesh", 22)
```

20.6 self

self refers to the **current object**.

class Student:

```
def __init__(self, name):  
    self.name = name
```

```
def display(self):  
    print(self.name)
```

Here:

self.name → attribute of the current object

self is a naming convention used for the instance reference. Python passes the instance automatically when an instance method is called.

20.7 Four Pillars of OOP

The four main principles are:

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
-

1. Encapsulation

Encapsulation means combining data and methods inside a class and controlling access to internal data.

class BankAccount:

```
def __init__(self, balance):  
    self.__balance = balance
```

__balance is treated as private through name mangling.

2. Inheritance

Inheritance allows one class to reuse properties and methods of another class.

```
class Animal:
    def eat(self):
        print("Eating")
```

```
class Dog(Animal):
    pass
```

Dog inherits from Animal.

3. Polymorphism

Polymorphism means the same interface can behave differently for different objects.

```
class Dog:
    def sound(self):
        print("Bark")
```

```
class Cat:
    def sound(self):
        print("Meow")
```

Both classes have sound(), but behavior is different.

4. Abstraction

Abstraction means hiding implementation details and exposing only the required functionality.

Example:

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass
```

20.8 Instance Variables

Variables belonging to individual objects.

class Student:

```
def __init__(self, name):  
    self.name = name
```

Each object can have a different name.

20.9 Class Variables

Variables shared by the class and, unless shadowed, accessible through its instances.

class Student:

```
college = "ADCE"  
  
def __init__(self, name):  
    self.name = name
```

college is a class variable.

20.10 Instance Methods

Methods that operate on an instance and receive self.

class Student:

```
def display(self):  
    print("Student")
```

20.11 Class Methods

A class method works with the **class**, rather than a particular instance.

It uses @classmethod and receives cls.

class Student:


```
college = "ADCE"
```

```
@classmethod
def change_college(cls, name):
    cls.college = name
```

20.12 Static Methods

A static method does not receive self or cls automatically.

It is generally used for utility logic related to the class.

class Calculator:

```
@staticmethod
def add(a, b):
    return a + b
```

```
print(Calculator.add(10, 20))
```

Instance vs Class vs Static Method

Method	First parameter	Used for
Instance method	self	Object data
Class method	cls	Class data
Static method	None	Utility logic

20.13 @classmethod

Decorator used to create a class method.

```
@classmethod
def method(cls):
    pass
```

20.14 @staticmethod

Decorator used to create a static method.

```
@staticmethod
def method():
```

pass

20.15 Access Modifiers

Python does not enforce Java/C++-style access modifiers in the same way.

Common conventions:

public → name
protected → _name
private → __name

Example:

class Student:

```
name = "Amit"  
_age = 22  
__marks = 90
```

20.16 Property Decorators

@property allows a method to be accessed like an attribute.

class Student:

```
def __init__(self, name):  
    self._name = name  
  
    @property  
    def name(self):  
        return self._name
```

Usage:

```
student = Student("Prathamesh")
```

```
print(student.name)
```

No () is required.

Properties are commonly used to control access to attributes.

Module 21: Inheritance

Inheritance allows a child class to acquire attributes and methods from a parent class.

21.1 Single Inheritance

One child inherits from one parent.

```
class Animal:
    def eat(self):
        print("Eating")
```

```
class Dog(Animal):
    pass
Animal
↓
Dog
```

21.2 Multiple Inheritance

One child inherits from multiple parent classes.

```
class Father:
    def skills(self):
        print("Driving")
```

```
class Mother:
    def talent(self):
        print("Cooking")
```

```
class Child(Father, Mother):
    pass
Father —┐
         └─> Child
Mother —┐
```

21.3 Multilevel Inheritance

Inheritance occurs across multiple levels.

Prathamesh Arvind Jadhav

```
class Grandparent:  
    pass
```

```
class Parent(Grandparent):  
    pass
```

```
class Child(Parent):  
    pass  
Grandparent  
    ↓  
Parent  
    ↓  
Child
```

21.4 Hierarchical Inheritance

Multiple child classes inherit from the same parent.

```
class Animal:  
    pass
```

```
class Dog(Animal):  
    pass
```

```
class Cat(Animal):  
    pass  
    Animal  
    /  \  
Dog   Cat
```

21.5 Hybrid Inheritance

A combination of two or more inheritance types.

Example:

```
    A  
  /\   
B  C  
  \/
```

D

Python supports such inheritance patterns.

21.6 Method Overriding

When a child class provides its own implementation of a parent method.

```
class Animal:
```

```
    def sound(self):
        print("Animal sound")
```

```
class Dog(Animal):
```

```
    def sound(self):
        print("Bark")
```

Dog overrides sound().

21.7 super()

super() is used to access functionality from a parent class.

```
class Animal:
```

```
    def __init__(self, name):
        self.name = name
```

```
class Dog(Animal):
```

```
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

Here:

```
super().__init__(name)
```

calls the parent constructor.

21.8 Method Resolution Order — MRO

MRO defines the order in which Python searches classes for methods and attributes.

Use:

```
ClassName.mro()
```

or:

```
ClassName.__mro__
```

Example:

```
class A:  
    pass
```

```
class B(A):  
    pass
```

```
print(B.mro())
```

Conceptually:

$B \rightarrow A \rightarrow \text{object}$

Python uses the **C3 linearization algorithm** to determine MRO.

Module 22: Polymorphism

Polymorphism means "many forms". The same operation/interface can work with different types or objects.

22.1 Method Overriding

Different child classes can implement the same method differently.

```
class Dog:
```

```
def sound(self):  
    print("Bark")
```

```
class Cat:
```

```
    def sound(self):  
        print("Meow")
```

```
for animal in [Dog(), Cat():  
    animal.sound()
```

Output:

Bark
Meow

The same:

```
animal.sound()
```

produces different behavior.

22.2 Duck Typing

Python often follows the principle:

"If it behaves like the required type, use it."

Example:

```
class Dog:
```

```
    def speak(self):  
        print("Bark")
```

```
class Person:
```

```
    def speak(self):  
        print("Hello")
```

```
def make_speak(obj):  
    obj.speak()
```

```
make_speak(Dog())  
make_speak(Person())
```

The function does not care about the object's class. It only cares that the object provides speak().

22.3 Operator Overloading

Python allows operators to be customized for objects using special methods.

Example:

class Number:

```
    def __init__(self, value):  
        self.value = value  
  
    def __add__(self, other):  
        return Number(self.value + other.value)
```

Now:

```
a = Number(10)  
b = Number(20)
```

```
c = a + b
```

+ internally uses:

```
__add__()
```

Common special methods:

```
+    → __add__()  
-    → __sub__()  
*    → __mul__()  
==   → __eq__()  
<    → __lt__()  
>    → __gt__()
```

22.4 Method Overloading Concept

In languages such as Java and C++, you can define multiple methods with the same name but different parameter lists.

Python does **not** support traditional method overloading by signature in the same way.

Example:

class Calculator:

```
def add(self, a, b):  
    return a + b
```

Defining another add() replaces the previous definition.

Python commonly achieves similar behavior using:

- Default arguments
- *args
- **kwargs
- Type checks when necessary

Example:

```
def add(a, b=0):  
    return a + b
```

22.5 Python's Approach to Polymorphism

Python uses several mechanisms:

Polymorphism

- Method overriding
- Duck typing
- Operator overloading
- Built-in functions
- Flexible function arguments

Example:

```
print(len("Python"))  
print(len([1, 2, 3]))
```

The same len() function works with different object types.

Module 23: Abstraction & Encapsulation

Part A: Abstraction

Abstraction means hiding implementation details and exposing only the necessary interface.

Example:

You use:
car.start()

You don't need to know:
How the engine internally starts.

23.1 Abstract Classes

An **abstract class** provides a common interface for subclasses and generally should not be instantiated directly.

Python provides abstract base classes through the abc module.

23.2 ABC

ABC is a base class used to define abstract base classes.

```
from abc import ABC
```

```
class Vehicle(ABC):  
    pass
```

23.3 abstractmethod

@abstractmethod marks a method that subclasses are required to implement before they can be instantiated.

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
```

```
@abstractmethod  
def start(self):  
    pass
```

Child class:

```
class Car(Vehicle):  
  
    def start(self):  
        print("Car started")
```

Now:

```
car = Car()  
car.start()
```

Output:

Car started

Trying to instantiate Vehicle() directly raises an error because it has an abstract method.

Part B: Encapsulation

Encapsulation means bundling data and methods together and controlling access to internal implementation details.

23.4 Public Members

Public members can be accessed normally.

class Student:

```
    def __init__(self):  
        self.name = "Prathamesh"
```

Usage:

```
student = Student()  
  
print(student.name)
```

23.5 Protected Convention `_variable`

A single underscore indicates that a member is intended for **internal/protected use by convention**.

class Student:

```
def __init__(self):  
    self._marks = 90
```

Python does not strictly prevent access:

```
print(student._marks)
```

The underscore is mainly a convention.

23.6 Private Convention `__variable`

A double underscore triggers **name mangling**.

class Student:

```
def __init__(self):  
    self.__marks = 90
```

Direct access:

```
student.__marks
```

does not normally work.

23.7 Name Mangling

Python changes the internal name of a double-underscore attribute.

```
__marks
```

inside class Student becomes approximately:

```
_Student__marks
```

So:

```
print(student._Student__marks)
```

can access it, although this bypasses the intended encapsulation.

Important

Python's private members are **not truly private** in the same way as some languages.

Module 24: Iterators & Generators

Iterators and generators are important for processing data one item at a time, especially when working with large datasets.

Part A: Iterators

24.1 Iterable

An **iterable** is an object that can be iterated over.

Examples:

list
tuple
string
set
dictionary

Example:

```
numbers = [1, 2, 3]
```

```
for number in numbers:  
    print(number)
```

A list is iterable.

24.2 Iterator

An **iterator** is an object that produces values one at a time using `next()`.

Create an iterator using:

```
iter()
```

Example:

```
numbers = [1, 2, 3]
```

```
iterator = iter(numbers)
```

```
print(next(iterator))  
print(next(iterator))  
print(next(iterator))
```

Output:

```
1  
2  
3
```

24.3 `iter()`

`iter()` converts an iterable into an iterator.

```
numbers = [10, 20, 30]
```

```
iterator = iter(numbers)
```

Now:

```
next(iterator)
```

gets the next value.

24.4 `next()`

`next()` retrieves the next item from an iterator.

```
numbers = [10, 20, 30]
```

```
it = iter(numbers)
```

```
print(next(it))  
print(next(it))
```

Output:

```
10  
20
```

When there are no more values, next() raises:

StopIteration

24.5 __iter__()

An iterator object implements:

`__iter__()`

It returns the iterator itself.

24.6 __next__()

An iterator implements:

`__next__()`

It returns the next value.

Example:

class Counter:

```
def __init__(self, limit):  
    self.current = 0  
    self.limit = limit  
  
def __iter__(self):  
    return self  
  
def __next__(self):  
    if self.current < self.limit:  
        value = self.current
```

```
self.current += 1  
return value
```

```
raise StopIteration
```

Usage:

```
counter = Counter(3)
```

```
print(next(counter))  
print(next(counter))  
print(next(counter))
```

Output:

```
0  
1  
2
```

Iterable vs Iterator

Iterable	Iterator
Can be iterated over	Produces next values
Can create an iterator	Has <code>__next__()</code>
Example: list	Example: <code>iter(list)</code>
<code>iter()</code> can be called on it	<code>next()</code> can be called on it

Easy way to remember

Iterable → Can be looped over

Iterator → Gives next item

Part B: Generators

24.7 Generator Functions

A **generator function** is a function that uses `yield` to produce values one at a time.

```
def numbers():  
    yield 1  
    yield 2  
    yield 3
```


Calling it:

```
gen = numbers()
```

creates a generator object.

Values are produced when requested:

```
print(next(gen))  
print(next(gen))  
print(next(gen))
```

Output:

```
1  
2  
3
```

24.8 yield

yield produces a value and **pauses the function**, preserving its state so execution can continue later.

Example:

```
def count():  
    yield 1  
    yield 2  
    yield 3
```

Unlike return, yield does not permanently terminate the generator on the first value.

return vs yield

return → Returns a value and ends the function
yield → Produces a value and pauses the generator

24.9 Generator Expressions

Generator expressions are similar to list comprehensions but use parentheses.

List comprehension

```
squares = [x * x for x in range(5)]
```

Generator expression

```
squares = (x * x for x in range(5))
```

The generator produces values as needed.

24.10 Generator vs Iterator

A **generator** is a special type of **iterator** created using a generator function or generator expression.

Iterator
↑
Generator

A generator automatically handles much of the iterator protocol for you.

24.11 Memory Efficiency

Generators are memory-efficient because they produce values **lazily**, rather than storing all values at once.

Example:

```
def numbers():  
    for i in range(1000000):  
        yield i
```

The generator does not create a list containing one million values in memory.

It generates each value when needed.

Useful for

- Large datasets
- Large files
- Data processing
- Streaming data
- Memory-sensitive applications